Modern Application Development I - Project Report

Trekking Operations Management System (TOMS)

1. Author

Name: Nikhil Kumar Shah
Roll Number: 25f2006470
Student Email: 25f2006470@ds.study.iitm.ac.in

About Me:
I am a student of the IIT Madras BS Degree Programme with an interest in software development and data-driven applications. I enjoy building practical web applications with a focus on usability, backend development, and reliable system design.

2. AI / LLM Usage

AI/LLM Used: Qwen2.5-Coder-7B-Instruct | **Estimated Usage:** Approximately 10%.
Used selectively for authentication, frontend/backend implementation support, debugging, validation, and testing; architecture, database design, business logic, integration, and final verification were performed manually.

3. Description

TOMS is a multi-role web application for managing trekking operations involving Administrators, Trek Staff, and Trekkers. It supports trek management, staff assignment, booking, availability control, and operational status tracking.

4. Technologies Used

Technology / Library	Purpose in Project
Python	Primary programming language for application logic and business rules
Flask	Backend framework for routing, controllers, and HTTP request handling
SQLite	Relational database for persistent application data
SQLAlchemy	ORM for database models, relationships, and queries
Jinja2	Server-side template engine for dynamic HTML rendering
Flask-Login	User authentication, session management, and role-based access
Flask-WTF / WTForms	Form handling, validation, and CSRF protection
Bootstrap 5	Responsive layout and reusable UI components
Werkzeug	Password hashing and Flask security utilities
Font Awesome 6	Icons for navigation, buttons, status indicators, and UI elements
HTML5 / CSS3	Page structure, forms, custom styling, and responsive presentation

5. DB Schema Design

The database consists of eight entities: Users, Treks, Bookings, Trek Staff Assignments, Trek Attendance, Trek Certificates, Trek Status Logs, and Notifications.

Table	Purpose	Key Relationships
Users	Accounts, roles and profile information	Creates Treks; places Bookings; receives Notifications
Treks	Trek details, capacity and status	Has Bookings and Staff Assignments
Bookings	Trek reservations and booking status	Links Users and Treks
Staff Assignments	Staff-to-trek mapping	Links Staff Users and Treks
Trek Attendance	Participant attendance/check-in	Linked to Bookings
Trek Certificates	Completion certificate records	Linked to Bookings
Trek Status Logs	Trek status change history	Linked to Treks and Users
Notifications	User notifications	Linked to Users

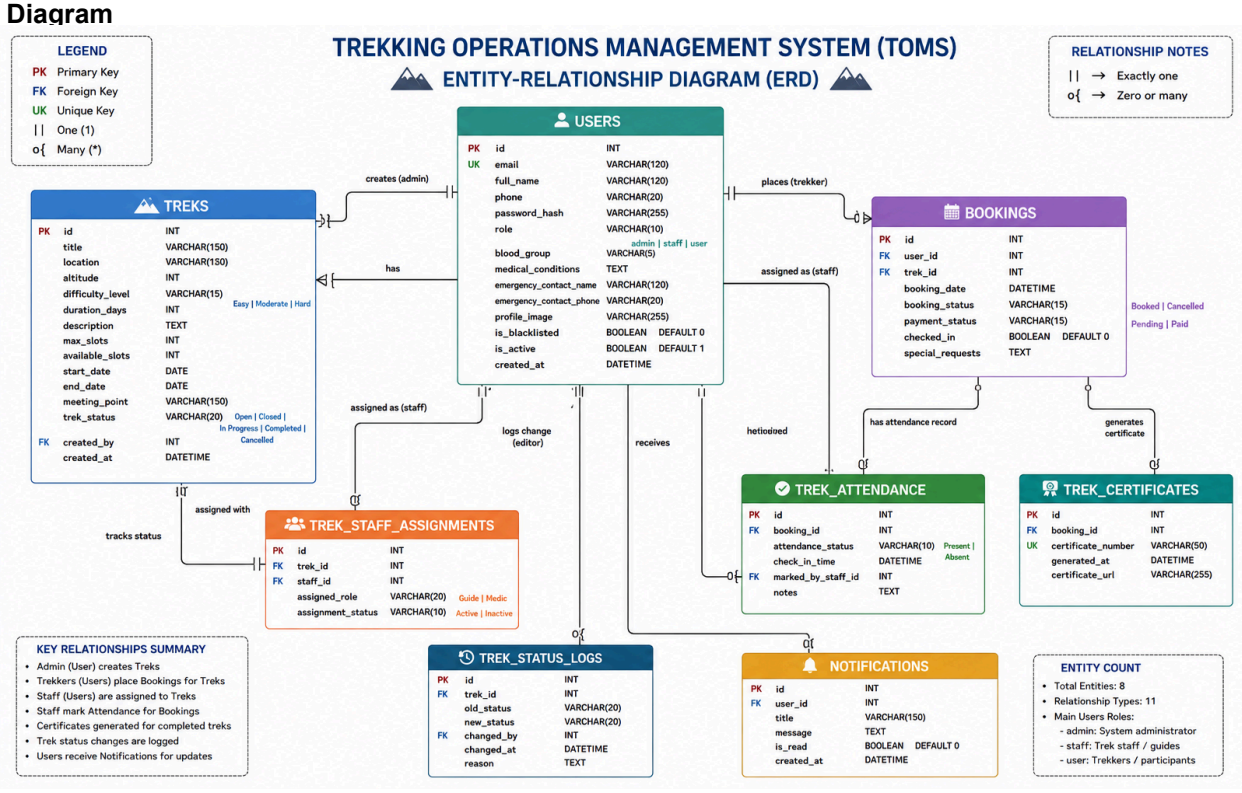
Key Relationships

User–Trek, User–Booking, User–Staff Assignment, User–Notification, Trek–Booking, Trek–Staff Assignment, Trek–Status Log, Booking–Attendance, and Booking–Certificate.

Design Rationale

The schema separates core operational entities from supporting records such as attendance, certificates, notifications, and status logs. This maintains clear relationships, avoids duplication, and supports the application's operational and audit requirements.

ER Diagram



6. API Design

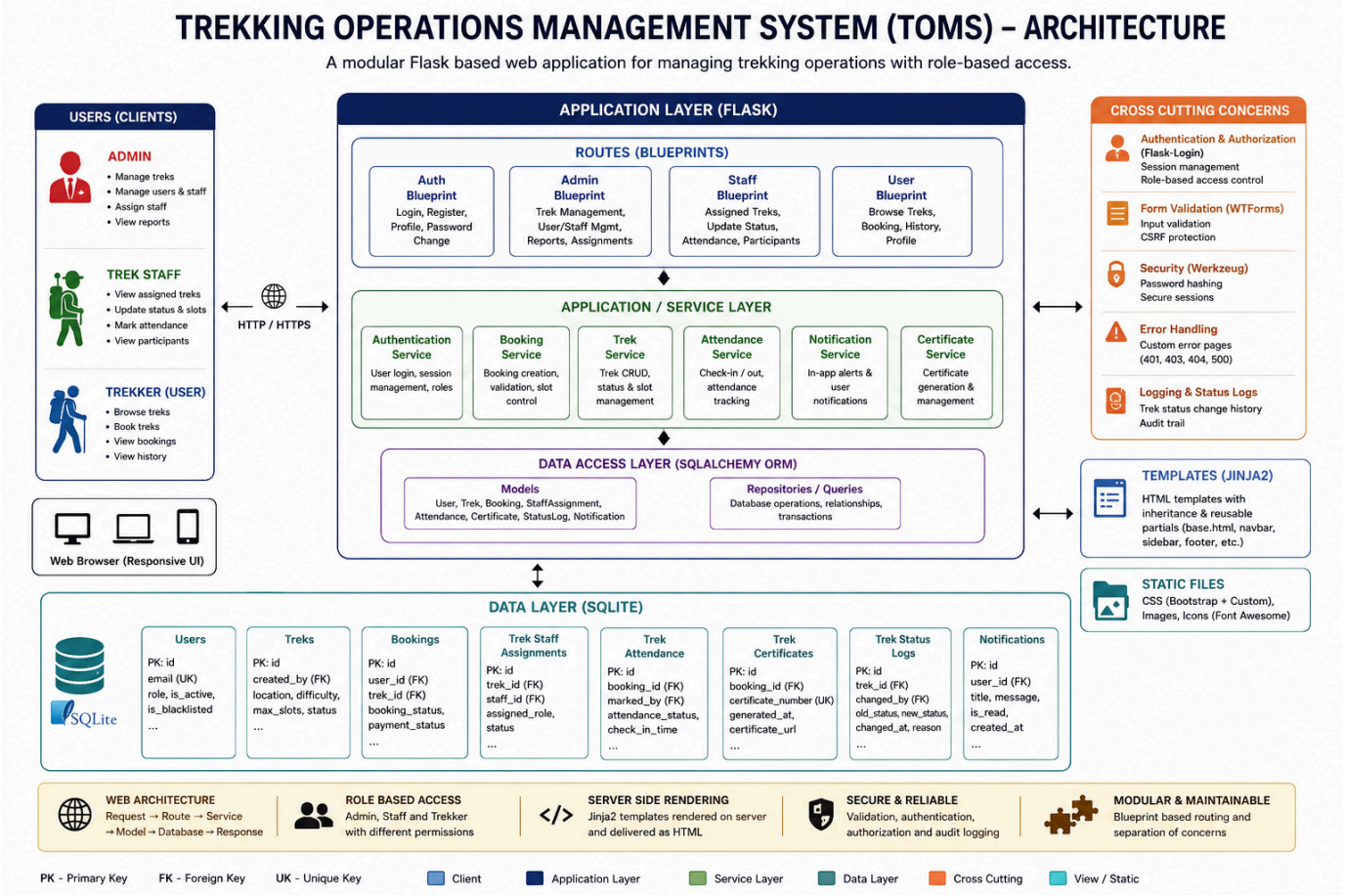
No separate REST API layer was implemented. The application uses Flask routes for browser-based authentication, trek management, booking, staff operations, and user workflows. API resources were optional and were not required for the core application.

Method	Endpoint	Purpose
POST	`/login`	Authenticate user
POST	`/register`	Register user/staff
GET	`/treks`	View available treks
POST	`/treks/<id>/book`	Create booking
POST	`/admin/staff/<id>/approve`	Approve staff
POST	`/admin/users/<id>/blacklist`	Manage user
POST	`/staff/treks/<id>/update-status`	Update trek status

The API routes perform request validation, authentication and authorization checks, business-rule validation, and database operations through the application layer.

7. Architecture and Features

Architecture



Implemented Features

Authentication and Role-Based Access Control:
Users can authenticate through role-specific access controls for Admin, Staff, and Trekker functionality.

Trek Management:
Administrators can create and manage treks including location, difficulty, capacity, dates, meeting point, and operational status.

Staff Management:
Staff registration and approval are managed through an administrative workflow, with staff assigned to specific trekking operations.

Booking Management:
Trekkers can browse available treks and create bookings subject to availability and operational rules.

Availability Control:
The booking workflow validates available slots and trek status to prevent invalid or over-capacity bookings.

Trek Operations:
Staff can manage assigned trekking activities and update operational status.

User Administration:
Administrators can search, manage, approve, and blacklist users according to system rules.

Security:
Passwords are stored using secure hashing, while authentication, sessions, validation, and role checks protect restricted operations.

Additional Features:
Participant attendance/check-in, completion certificates, in-app notifications, extended user profiles, booking payment-status tracking, and trek status history/audit logs.

8. Video Presentation

Video Link: Modern Application Development 1 Project Video.mp4 or https://drive.google.com/file/d/1xVXnwtOAK5IGO1F_pjHr7xyMN_2uuvSS/view?usp=sharing